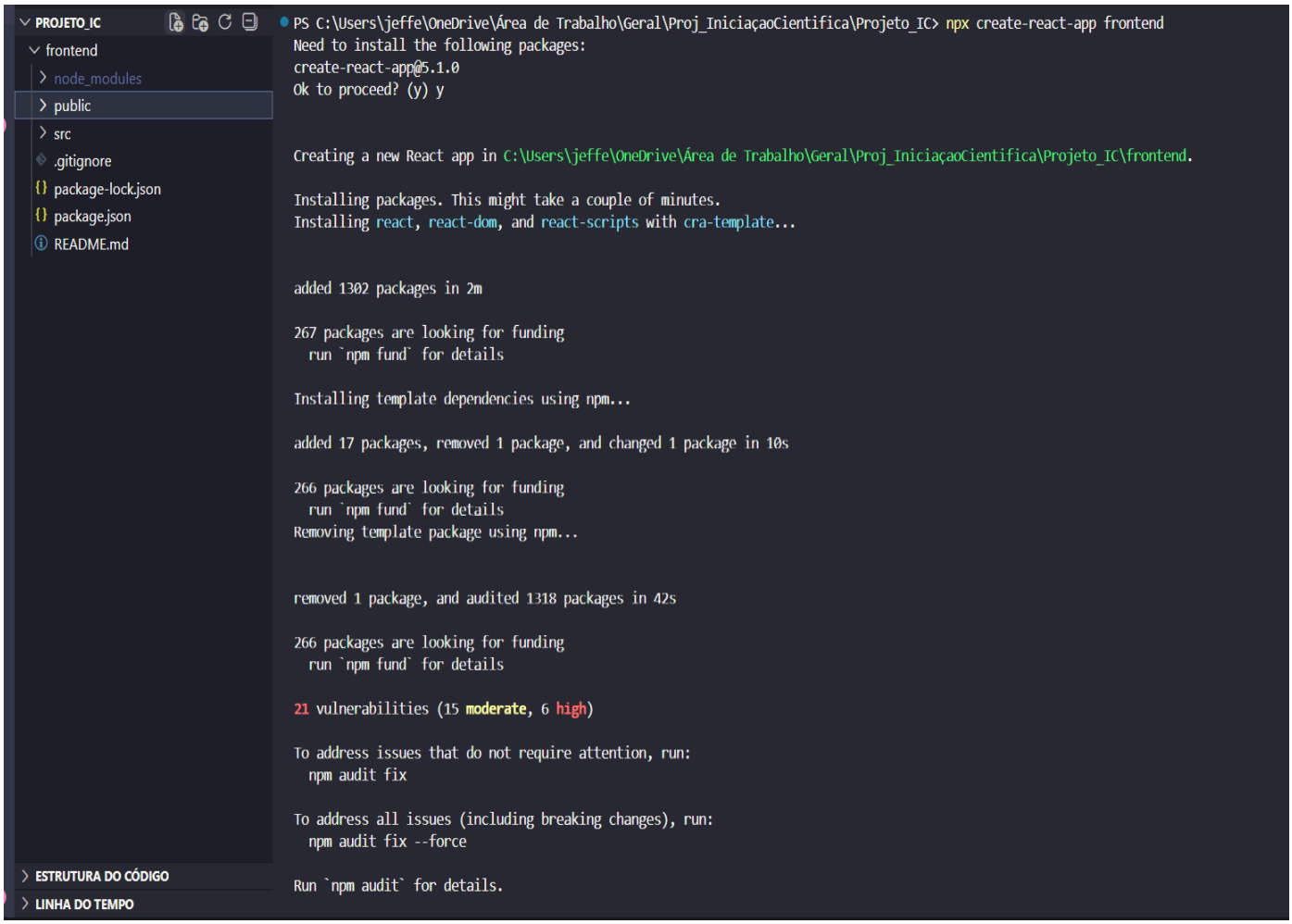
Versões utilizadas no projeto (fase atual):

- Node.js: v20.20.0
- React: v19.2.4

Data: 02/02

Para começar, criei a pasta Projeto_IC, entrei no terminal e digitei:

npx create-react-app frontend



Depois, entrei na pasta que criei (**cd frontend**) e rodei o comando **npm star**t para abrir a aplicação no navegador em localhost:3000.

OBS: {Sobre o arquivo **src/reportWebVitals.js**, ele não é essencial para rodar a aplicação React. Porém, esse arquivo é útil se quisermos monitorar a performance real da aplicação em produção, coletando métricas de experiência do usuário. Decidi manter ele, falar com o Evandro depois para decidir se vamos utilizá-lo.}

Dentro da pasta **src**, criei a pasta **components**. Dentro dela, criei a pasta **layout** e, dentro dessa pasta, o arquivo **MainLayout.jsx**.

OBS: {Hoje, em aplicações React, não existe mais um arquivo .html separado para cada tela. Em vez disso, utiliza-se o JSX, que nada mais é do que JavaScript capaz de retornar algo muito parecido com HTML}

Dentro do MainLayout.jsx, comecei importando o **Sidebar.jsx**, que será o nosso menu lateral. Também utilizei algo chamado **children ({children})** dentro da tag . Esse children funciona como um espaço vazio que será preenchido pelo App.js (ou qualquer outro componente que use o MainLayout). Tudo o que for escrito dentro de ... aparece exatamente no lugar do {children}.

Data: 03/02

`O App.js é o orquestrador da aplicação.`

Depois de criar o menu lateral, antes de partir para o menu superior, precisei deixar as opções (como “Zero de Funções”) clicáveis e armazenar essa escolha em uma variável.

Pensei inicialmente em guardar esse estado no Sidebar.jsx, mas percebi que o ideal é manter o estado no App.js, já que o estado deve ficar no nível mais alto que precisa dele.

No momento, o CSS do Sidebar está dentro do próprio arquivo Sidebar.jsx. Mais para frente, pretendo criar um arquivo Sidebar.css separado apenas para estilização.

Data: 04/02

Relacionando com o do dia anterior:

import { useState } from "react";

O import { useState } from "react" traz apenas o hook useState da biblioteca React para dentro do arquivo. O useState permite criar variáveis de estado dentro de componentes funcionais, ou seja, valores que podem mudar ao longo do tempo e que, quando atualizados pela função correspondente (ex: setCategoriaSelecionada), fazem o componente ser renderizado novamente automaticamente.

O useState(null) retorna um array com dois itens:

1. **O valor atual do estado (inicialmente null).**
2. **A função que atualiza esse valor.**

A linha:

const [categoriaSelecionada, setCategoriaSelecionada] = useState(null)

utiliza desestruturação do JavaScript para separar esses dois valores.

- Ele pega o **primeiro item** do array e coloca em `categoriaSelecionada`.

- Ele pega o **segundo item** do array e coloca em `setCategoriaSelecionada`.

Antes, o Sidebar estava assim:

```
<Sidebar />
```

OBS: Ou seja apenas renderizava o menu lateral. Depois que criei a lógica de categoria, passei uma prop para o Sidebar:

Agora que criei a categoria

Depois que criei a lógica de categoria, passei uma prop para o Sidebar:

```
<Sidebar aoSelecionarcategoria={aoSelecionarcategoria} />
```

Porém o MainLayout não tem acesso direto ao estado. então precisei passar essa função primeiro para o MainLayout:

```
<MainLayout aoSelecionarcategoria={setCategoriaSelecionada}>
```

E depois recebê-la como parâmetro no MainLayout.jsx:

function MainLayout({ *children* , *aoSelecionarcategoria*}{...

OBS: Componentes filhos não alteram diretamente o estado global. Eles apenas avisam que algo aconteceu, como: “alguém clicou aqui”.

No arquivo Sidebar.jsx, passei a receber essa função:

```
function Sidebar({ aoSelecionarcategoria }) {
```

Assim, o Sidebar passa a conhecer a função que veio do App.js, que é o nosso orquestrador.

Em cada item do menu lateral, adicionei um onClick para chamar essa função, passando o nome da categoria selecionada:

Depois disso, comecei a planejar o menu superior, que só deve aparecer quando o usuário selecionar uma opção no menu lateral.

> tenho que criar um arquivo menu superior. jsx no mesmo caminho que os outros, fazer ele receber a categoria selecionada. (var categoriaselecionad))
>
> e daí adicionar alguns botões sobre cada método que pode ser feito para aquela conta(exemplo: usuário selecionou zero de funções no menu lateral) vai abrir o menu superior com dois botões com Bisseção e Newton ele apertando em algum botão. A tela abaixo irá…?

Data: 06/02

Criei o arquivo **TopMenu.jsx** só que diferente do **sidebar.jsx (menulateral)** eu nao instanciei ele dentro do **mainLayout.jsx** importando ele no começo.

Eu adicionei ele dentro da tag **<main Layout>** dentro do **app.js** pelo fato que o topmenu nao vai ser algo fixo na tela, igual o menu lateral.

OBS: Acredito que pensei certo nisso. **Esta?**

OBS:

- O Sidebar **sempre existe**
- Ele faz parte da **estrutura da aplicação**
- Independente do que o usuário escolher o Sidebar estará lá

Dentro do **topmenu** comecei criando um **IF** para ver qual categoria foi selecionada no **sidebar**.

Dentro do TopMenu, comecei criando uma condição para verificar qual categoria foi selecionada no Sidebar. Inicialmente implementei apenas para “Zero de Funções”, exibindo dois botões: Bisseção e Newton.

Após isso vou colocar as verificações para integrais e interpolação, adicionar os botões para integrais e interpolação e pretendo implementar as categorias de Interpolação e Integrais e, por fim, organizar melhor o CSS do projeto.

OBS:

Sidebar seleciona Categoria
TopMenu seleciona Método
App decide o que vai renderizar

Dúvidas matemáticas:

Interpolação

A interpolação serve para encontrar valores intermediários entre pontos conhecidos.

> **Polinômios de Lagrange:** É a forma clássica de ajustar um polinômio que passa por todos os pontos. Muito didático, mas pode sofrer com oscilações nas bordas (Fenômeno de Runge).
>
> **Splines Cubícolas:** De longe a mais utilizada na prática. Em vez de um único polinômio complexo, ela usa vários polinômios de grau 3 conectados. É o que softwares de design e animação usam para criar curvas suaves.

Polinômios de Newton: Excelente quando você precisa adicionar mais pontos ao conjunto de dados sem ter que refazer todo o cálculo do zero, graças às

Diferenças Divididas (?).

Integrais

Integração Numérica (Onde a conta de verdade acontece)

Basicamente, a gente usa isso quando a integral é muito difícil.

Regra do Trapézio: É a mais "feijão com arroz". Você finge que o gráfico é um monte de trapézios e soma a área deles. Não é a mais precisa do mundo (ela corta as curvas com retas, então perde um pouco de informação)

Regra de Simpson (1/3 e 3/8): Essa aqui em vez de ligar os pontos com retas, ela usa curvas (parábolas). E com quase o mesmo esforço do trapézio, conseguimos um resultado muito mais próximo do real.

Quadratura Gaussiana: Enquanto nas outras usamos pontos com a mesma distância entre eles, aqui escolhe pontos estratégicos (que não são óbvios) para chegar no resultado com muito menos conta.

Data: 10/03 e 11/03

Antes eu só tinha feito (teste) com o “Zero de funções” e aparecia dois botões: “Bisseção” e “Newton”.

Agora fiz uma melhoria para aparecer os métodos da categoria selecionada pelo usuário, para isso a minha ideia foi em vez de colocar vários IFs, fiz uma breve estrutura (Objeto) que daí associa cada categoria ao seus métodos.

```
const metodosDisponiveis = {
    "Zero de Funções": ["Bisseção", "Newton"],
    "Interpolação": ["Lagrange", "Newton"],
    "Integrais": ["Trapézio", "Simpson"]
}
```

Assim, quando a categoria é selecionada, o sistema busca no objeto os métodos correspondentes. Caso não existir não exibe nada

pra isso eu utilize a função .map() que vai passar por todos os elementos do nosso array (vetor metodosDaCategoria) e caso a gente precisar adicionar mais métodos fica bem fácil.